One Strategic Path Forward: Resolving Temporal Paradoxes Through Certified State Generation

The Epistemological Crisis of Predictive Drafting

The foundational governance mechanism of the Stela platform relies upon the Policy of Truth (PoT), a singular constitutional document that mandates strict deterministic operational behavior across all system participants. Under the epistemic standards formally defined in PoT.md §4.5, the definition of system truth is absolute and uncompromising: everything the system proves is actually true, truth itself is a primitive predicate governed by specific rules rather than a mere theoretical definition, and any introduction of deception or incoherence is inherently destabilizing and must be resolved immediately to maintain operational integrity.[1] However, a critical systemic vulnerability has been identified within the Dispatch Packet (DP) drafting workflow, leading to an architectural crisis defined mathematically and logically as the "temporal paradox".[1]

The temporal paradox occurs when an Integrator—the Lead AI responsible for maintaining system governance, structural integrity, and auditing—attempts to author the final "Closing Block" of a Dispatch Packet before the implementation phase has been executed by a Contractor, the Guest AI tasked with code generation.[1] This Closing Block typically contains required execution metadata such as final git commit hashes, Pull Request descriptions, exact file path modifications, and complex verification command statistics.[1] By forcing the Integrator to predict the exact future state of the repository before that state exists, the system inherently requires the Integrator to hallucinate probabilistic outcomes, generating "ghost pointers" to artifacts that have not yet been produced.[1]

This predictive drafting represents a catastrophic violation of the core tenets of the Stela infrastructure. When the Integrator guesses the downstream effects of an unexecuted plan, it mechanically introduces "Canon Drift." Canon Drift is explicitly defined in PoT.md §1.2 and §2 as any divergence between canon and repository state, or the duplication of canon outside the Single Source of Truth (SSOT). This drift immediately triggers a System Failure State requiring a hard stop and the outright rejection of pull requests.[1] The temporal paradox generates false proof pointers, undermines the cryptographically secure integrity of the SoP.md (State of Play) and PoW.md (Proof of Work) ledgers, and fundamentally degrades the absolute literalism required by the platform's Behavioral Logic Standard.[1]

To permanently restore epistemological equilibrium and mathematically eliminate the temporal paradox, a unified strategic path forward has been synthesized from proposed architectural plans. This strategy mandates a comprehensive and irreversible transition from the flawed

"Authored Future State" model to a highly deterministic "Certified State Generation" model, heavily grounded in Correct-by-Construction (CbC) methodologies.[1] The ensuing analysis details the rigorous, phased operational directives, architectural realignments, and structural toolchain enhancements required to achieve this terminal, self-healing system state.

The Correct-by-Construction (CbC) Paradigm Shift

The definitive resolution to the predictive drafting crisis requires moving away from post-execution detection architectures—such as executing linting scripts to identify hallucinated file paths only after they have been erroneously authored—and toward the implementation of absolute structural prevention.[1] The Correct-by-Construction (CbC) paradigm dictates that invalid system states, including predictive metadata generation, should be made mechanically impossible to instantiate at the point of origin.[1]

Template Re-engineering and Structural Prevention

The primary mechanism enabling the temporal paradox is the existence of predictive slot variables within the initial drafting template. To enforce structural prevention, the static Closing Block slots—specifically the Extended Technical Manifest, Final Squash Stub, Primary Commit Header, and Pull Request Description—must be physically and irrevocably severed from the initial DP drafting template located at ops/src/surfaces/dp.md.tpl.[1]

By removing these output-dependent fields from the TASK.md contract surface during the draft phase, it becomes structurally impossible for the Integrator to populate them with probabilistic guesses.[1] The Integrator is thus confined strictly to authoring the objective, scope, safety constraints, and execution plan.[1] The generation of evidence is strictly decoupled from the generation of intent. The ops/bin/draft binary, which generates the canonical DP structure, ensures that manual structural edits to the boilerplate are prohibited, guaranteeing that the predictive slots cannot be manually reintroduced by an erring Integrator.[1]

This architectural shift aligns seamlessly with the CbC Design Discipline Preflight checks now mandated in the repository.[1] As codified in the operational directives and verified through TASK.md §3.1.1, whenever the system architecture is modified, the design must evaluate whether a structural prevention is available to replace a behavioral rule.[1] In this application, modifying the dp.md.tpl template to exclude output-dependent fields represents the ultimate structural prevention, satisfying the highest tier of the CbC deletion test. As evidenced by historical design decisions, such as the DP-OPS-0101 CbC Phase 1 Canonization, enforcement logic must be built into the templates and generation binaries themselves, rather than relying solely on downstream linting scripts that act as mere behavior fighters.[1]

Enforcement of the Generation Mandate

PoT.md §4.2 establishes a hard, non-negotiable constraint known as the Generation Mandate:

"Verification outputs must be generated, not pre-filled or assumed" and specifically, "DP RESULTS receipts are generated artifacts produced by ops/bin/certify; manual fabrication is prohibited".[1] The CbC paradigm operationalizes this mandate. Metadata regarding the execution cannot be manually authored by any AI agent; it must be procedurally emitted by the ops/bin/certify binary upon analyzing the terminal git state of the execution branch.[1]

The shift to Certified State Generation ensures that the verification proofs appended to PoW.md are mathematically bound to the physical artifacts present in storage/handoff/, completely eliminating predictive hallucinations. If a Contractor attempts to circumvent this by manually assembling receipt content, the system will recognize the fabrication through missing cryptographic hashes or structurally malformed data, immediately triggering an Ambiguity or Dirty State System Failure.[1]

Phase 0: Disaggregation of Intent and the Elimination of Mixed Packets

The transition to Certified State Generation necessitates an immediate operational directive: the absolute cessation of mixed-mode Dispatch Packets.[1] Historically, Integrators have attempted to author DPs that combine investigative, exploratory tasks with deterministic, state-mutating implementation tasks within the same execution sequence. This practice induces severe ambiguity, violates strict allowlist governance, and frequently leads to System Failure States.

The Prohibition of Conditional Execution and "If/Then" Logic

In a mixed packet, the Integrator often relies on conditional (if/then) execution logic. For example, a directive might instruct the Contractor to "investigate the configuration, and *if* a specific error pattern is found, *then* rewrite the binary to mitigate it." This architectural pattern is fundamentally incompatible with the Stela Staffing Protocol established in PoT.md §4.1.[1] The Contractor (Guest AI) is authorized only to execute specific, pre-determined logic tasks and draft implementation details within a strictly defined, immutable scope.[1] The Contractor does not possess the algorithmic authority to make governance decisions, interpret broad intent, or autonomously expand the scope of its file modifications based on runtime discoveries.

Allowing the Contractor to evaluate conditional branches delegates decision-making authority downward, directly violating the axiom that the Operator (Human) exclusively owns final decisions and approvals.[1] Furthermore, conditional execution makes it impossible to guarantee the exact file paths that will be mutated prior to execution. This violates the hard constraint that every tracked mutation must pre-exist in the storage/dp/active/allowlist.txt file before execution begins.[1] If a Contractor executes an if branch that touches a file not explicitly authorized in the allowlist, the tools/lint/integrity.sh gate will detect the drift via git diff

--name-only and kill the process, halting all progress.[1]

The Investigate-Decide-Implement Pipeline

To permanently eliminate the reliance on conditional logic and ensure absolute literalism, all tasks requiring technical viability checks, code exploration, or structural discovery must be formally disaggregated into a minimum of two strictly bounded DPs, separated by an Operator decision review.[1]

Pipeline Stage	Operational Actor	Permitted Action	Constrained Output
1. Investigate	Integrator / Contractor	Read system state, analyze telemetry logs, trace execution paths.	A single decision record or data artifact. No state mutation.
2. Decide	Operator (Human)	Review empirical findings, approve recommendations, authorize scope.	Operator Authorization signature inside the decision record.
3. Implement	Contractor (Guest AI)	Execute deterministic logic based strictly on the approved parameters.	Deterministic file mutation constrained to the pre-approved allowlist.

Under this strict pipeline, if ambiguity arises during execution, the "Absolute literalism" mandate requires that instructions capable of multiple interpretations must trigger an immediate worker complaint and halt, mapping directly to the "Ambiguity" drift trigger defined in PoT.md §2.[1] The Contractor must cease execution rather than attempting to heuristically guess the Integrator's unstated intent, forcing the system back to the Operator for explicit parameter redefinition.[1]

Phase 1: Typological Protocol Enforcement and Boundary Isolation

With mixed-mode execution definitively eliminated, the strategic path forward formally introduces the Typological Split as a first-class routing protocol. This enforces two mutually exclusive DP topologies, ensuring the Integrator is never placed in a position requiring downstream outcome prediction and the Contractor is never forced to evaluate conditional

operations.[1]

Topology A: Investigative Dispatch Packets

Investigative DPs are engineered exclusively for discovery, system auditing, viability assessments, and the collection of operational telemetry.[1] They function as the sensory organs of the governance system, interacting with the repository strictly in a read-only capacity regarding production logic.

- **Jurisdictional Constraints:** The scope of an Investigative DP is entirely restricted from modifying any executable paths within the ops/, projects/, or tools/ directories.
- **Permitted Mutations:** The sole permitted mutation resulting from an Investigative DP is the creation of a solitary Decision Record generated within the archives/decisions/ directory.[1] As demonstrated by historical artifacts such as DEC-2026-02-23-001-certify-compat-0103.md, this record must adhere to a strict YAML frontmatter schema including trace_id, decision_id, packet_id, decision_type, and authorized_by.[1] The body of the document captures the empirical findings of the investigation and serves as the staging ground for Operator review.
- **Strict Prohibitions:** Investigative DPs are strictly prohibited from modifying payload code, altering runtime binaries, generating spec stubs, or creating speculative pull request metadata.[1] They must not attempt to predict what code *should* be written; their sole algorithmic mandate is to document what state currently *exists*.

Topology B: Deterministic Dispatch Packets

Deterministic DPs are engineered to enact approved, mathematically certain modifications to the repository, executing only the conclusions reached via prior Investigative DPs or direct Operator mandates.[1]

- **Jurisdictional Constraints:** The execution is strictly and irrevocably bound to the parameters pre-authorized by the Operator. The Integrator provides the exact execution logic required to achieve the outcome without requiring the Contractor to interpret abstract goals.
- **Permitted Mutations:** Mutations are constrained absolutely to the precise file paths enumerated in the active allowlist (storage/dp/active/allowlist.txt).[1] The tools/lint/integrity.sh script enforces this by utilizing the comm binary to cross-reference git diff --name-only against the sorted allowlist.[1] Any attempt by the Contractor to touch a file not present on this list, or any ghost file identified by git ls-files --others --exclude-standard, results in an immediate lint failure and a hard process kill.[1]
- **Strict Prohibitions:** Branching logic, conditional exploration, and automated fallback strategies are entirely forbidden.[1] If the Contractor encounters an unexpected state—for instance, if a requested variable target does not exist in the specified file, or if a regular expression replacement fails to match—the Contractor is prohibited from "fixing" the issue through unapproved heuristic searching. The worker must halt, emit a standard error log

to var/tmp/, and await Integrator/Operator remediation.

By rigidly adhering to these typologies, the Integrator's drafting burden is fundamentally simplified and stripped of predictive risk. When drafting an Investigative DP, the Integrator knows the only output will be a decision record in the archives/decisions/ subtree. When drafting a Deterministic DP, the Integrator knows the exact allowlist beforehand because the investigation phase has already concluded. The temporal paradox is thereby dissolved through structural isolation.

Phase 2: Decoupled Just-In-Time (JIT) Certification

The physical, mechanical resolution to the temporal paradox lies in decoupling the execution intent from the execution reality.[1] This is achieved by shifting the generation of narrative metadata away from the TASK.md DP drafting phase and relocating it to a Just-In-Time (JIT) certification phase occurring at the precise moment of execution completion, effectively moving from an *a priori* prediction model to an *a posteriori* observation model.[1]

The Relocation to the Handoff Sidecar

The narrative components previously hallucinated in the drafting phase are permanently relocated to a dedicated execution sidecar file.[1] During execution, the Contractor must actively maintain this human-authored Closing Block sidecar at the prescribed path: storage/handoff/CLOSING-DP-OPS-XXXX.md.[1]

Because the Contractor (or Operator) is authoring this file *during* or immediately after the implementation phase, it possesses absolute, real-time awareness of the modified state. It does not need to guess which files were touched because the git state provides empirical proof. The structural rules governing the authorship of this sidecar were hardened during the DP-OPS-0082 Documentation Enforcement Foundation and are strictly enforced by the tools/lint/style.sh gate to prevent degradation [1]:

1. **Syntactic Diversity:** Each of the six Mandatory Closing Block entries must utilize a distinct sentence structure. No two entries may open with the same lead verb or the same grammatical form, ensuring thoughtful authorship rather than automated boilerplate repetition.[1]
2. **Header Differentiation:** The Primary Commit Header must be an imperative clause and must not echo the Pull Request Title verbatim. At a minimum, the verb or the object must differ.[1]
3. **Markdown Validation:** The Pull Request Description field must contain at least one markdown construct (a heading beginning with ##, a list item, or bold text). If the heuristic detects a plain text paragraph, it flags the entry as an improper use of the reviewer interface and fails the gate.[1]
4. **Manifest Rigor:** The Extended Technical Manifest (plaintext) field must contain file paths only. The linting heuristic tokenizes the lines on whitespace; if two or more consecutive

tokens are present that do not begin with a letter or digit and do not contain a /, the line is classified as prose, and the style lint emits a hard failure. This guarantees absolute machine-readability for future archeological tools.[1]

5. **Conversational Utility:** The Review Conversation Starter must pose a genuine, open question ending in a question mark (?), targeting a specific design tradeoff or risk encountered during execution. Generic approval-seeking stems (e.g., "Does this look correct?") are mathematically rejected by the linter, forcing deep contextual engagement.[1]

The Upgrade of ops/bin/certify as the Absolute Truth Generator

With the sidecar maintaining the narrative reality, the ops/bin/certify binary serves as the absolute truth generator for the system.[1] It is the sole mechanism permitted to instantiate the RESULTS receipt; manual assembly of receipt content is a violation of the PoT and triggers an automatic process kill.[1] The JIT metadata generation process operates through a strict, deterministic sequence:

1. **Sidecar Ingestion:** The ops/bin/certify binary reads the storage/handoff/CLOSING-*.md sidecar to ingest the narratively authored fields.[1] It will initiate a hard stop if the file is missing or if any of the mandatory fields are empty.[1]
2. **Integrity and Path Validation:** The binary verifies the exact git state impact, comparing the output of git diff --name-only against the contents of storage/dp/active/allowlist.txt. Any deviation results in an immediate failure.[1]
3. **Deterministic Command Replay:** The binary executes the mandatory verification command list predefined in Section 3.4.5 of the DP. To ensure absolute determinism and prevent shell injection vulnerabilities, command substitution forms (e.g., expressions beginning with $()) are explicitly rejected by the certify literal replay engine. Only absolute or relative literal paths are accepted.[1] This rule is cross-enforced at lint time by tools/lint/dp.sh.
4. **Template Rendering:** The binary injects the ingested sidecar data, the precise diff statistics, and the command output logs into the ops/src/surfaces/results.md.tpl template to render the final DP-OPS-XXXX-RESULTS.md artifact.[1]
5. **Ledger Emittance:** Finally, ops/bin/certify generates the schema-stamped surface leaves for the archives/surfaces/ directory and safely rewrites the SoP.md and PoW.md single-line pointers to reference these newly minted, immutable cryptographic leaves.[1]

By relying exclusively on ops/bin/certify to synthesize the mathematical reality of the git state with the narrative context of the sidecar, the system achieves a 100% Correct-by-Construction output. The resulting RESULTS receipt is irrefutable, completely eliminating the epistemological decay caused by predictive drafting.

Constitutional Alignment with the Filing Doctrine

Implementing the Typological Split and JIT Certification models requires strict adherence to

the Physical Laws outlined in PoT.md §1.1. The Stela Filing Doctrine is a zero-tolerance architectural boundary that ensures payload, explanation, and execution layers are structurally isolated.[1]

Jurisdictional Boundaries and State Isolation

The repository is divided into discrete functional zones, and any cross-contamination is treated as a canonical drift failure:

- **ops/ (Run):** This directory is the exclusive domain of runtime binaries (ops/bin/), shared scripts (ops/lib/scripts/), and automation manifests.[1] The ops/bin/certify binary, the ops/bin/draft template generator, and the contextual bundle generator ops/bin/llms reside strictly here. Project payload code must never import from or cross-contaminate the ops/ namespace.[1]
- **docs/ (Explain):** This space contains manuals, operational rationales, and the authoritative registries (docs/ops/registry/). Under the CbC paradigm, docs/ is the only acceptable location for explanatory justification.[1] Binaries and scripts must not be placed in docs/, a rule actively enforced by tools/verify.sh which rejects any non-markdown artifact in this subtree.[1]
- **projects/ (Work):** The isolated environment for payload code. Deterministic DPs targeting projects must isolate their operational logic to this directory and adhere strictly to the project-specific rules defined in projects/STELA.md.[1]
- **storage/ (Payload):** This directory holds the artifacts of the execution lifecycle.[1] The active allowlist (storage/dp/active/allowlist.txt), the JIT sidecar (storage/handoff/CLOSING-*.md), and the final certified receipts (storage/handoff/DP-OPS-XXXX-RESULTS.md) are housed here. DPs are staged in storage/dp/intake/ and moved to storage/dp/processed/ post-execution.[1]
- **var/tmp/ (Resume):** The exclusive location for ephemeral task scratch space. Temporary file creation in unauthorized directories (such as the root /tmp) is strictly forbidden and triggers an immediate worker failure state as defined in the Contractor Briefs.[1]
- **archives/ (Cold):** Long-lived draft ledgers, decision records, and immutable surface leaves generated by ops/bin/compile and ops/bin/certify are permanently stored here.[1]

The Context Hazard Protocol and Verification Gates

The Filing Doctrine also dictates context consumption parameters. As Integrators draft Investigative or Deterministic DPs, they must obey the context hazard protocols. Tools like ops/bin/llms compile canonical surfaces into llms.txt, llms-core.txt, and llms-full.txt.[1] A git hook located at .github/hooks/llms ensures bundle freshness is enforced structurally before every commit.[1] However, dynamic, JIT-only definitions (such as draft skills in opt/_factory/skills/) are purposefully isolated and excluded from global context to prevent Integrators from hallucinating references to unpromoted tools.[1]

To ensure stability during the execution of Certified State Generation, the tools/verify.sh infrastructure gate acts as a sweeping enforcer of this doctrine.[1] It enforces the presence of required placeholders (e.g., storage/dumps/.gitkeep, var/tmp/.gitkeep), rejects non-markdown files in docs/ and opt/, and strictly verifies the integrity of factory promotion pointers, ensuring that candidate definitions resolve correctly under archives/definitions/.[1] Any drift in the physical layout is blocked before the DP can even enter the preflight phase, safeguarding the runtime environment.

Calibrated Canonical Density (CCD) and Linguistic Precision

A robust Certified State Generation architecture requires that the textual inputs dictating system state are parsed deterministically by both automated linters and human operators. The behavioral logic standard of the Stela platform relies heavily on Absolute Literalism and Linguistic Precision (PoT.md §4.2).[1] To enforce this, the platform utilizes Calibrated Canonical Density (CCD), a rigorous framework defining exact prose density parameters across different functional tiers.[1]

CCD Header Enforcement

As operationalized during the DP-OPS-0088 wave, canonical files are required to include a specialized HTML comment header immediately following the frontmatter: ``.[1] This header dictates the intended informational density of the document.

For example, PoT.md utilizes an ff_target="governance-narrative" with an ff_band="30-40", allowing for precise, legalistic prose.[1] Conversely, machine-facing templates like ops/src/surfaces/task.md.tpl demand an ff_target="machine-dense" with a highly restrictive ff_band="15-25", enforcing extreme brevity.[1] The tools/lint/ff.sh script programmatically parses these headers and scores the markdown content against the declared bands. If the prose is too sparse or too verbose, the linter emits a hard failure, forcing the author to compress or expand the text to meet the rigorous density standards.[1]

Absolute Literalism and Anti-Jargon Guardrails

The JIT certification model relies on the exact parsing of textual data, rendering linguistic ambiguity an unacceptable systemic risk. The Linguistic Precision mandate specifically prohibits the use of contractions across all documentation and execution scopes, including payload projects.[1] The use of contractions introduces parsing complexity for automated Natural Language Processing (NLP) validation tools and semantic ambiguity for operators analyzing incident logs.

This absolute literalism is actively enforced by tools/lint/truth.sh and tools/lint/style.sh.[1] The tools/lint/truth.sh script acts as a strict canon spelling integrity gate. It scans the repository for

embedded forbidden platform-name variants, forbidden legacy phrase tokens, and incorrect registry path-casing.[1] If a Contractor generating the execution sidecar uses a forbidden legacy token to describe a system state, the truth linter will identify the exact file and line number and return a non-zero exit code, blocking the ops/bin/certify sequence until absolute textual hygiene is restored.[1] This ensures that technical rationales remain rooted in observable system histories rather than theoretical jargon.

Ledger Equilibrium and the Audit Paradigm

The final phase of stabilizing the Certified State Generation strategy involves the continuous updating and strict harmonization of the platform's historical ledgers: SoP.md (State of Play) and PoW.md (Proof of Work).[1] These two documents represent the canonical memory of the system, yet their functions are entirely demarcated to prevent payload corruption and maintain unassailable auditability.

The History Ledger (SoP.md)

The SoP.md surface is strictly the history ledger.[1] It records the narrative context of what shipped and, critically, *why* it shipped.[1] No permanent rules or executable constraints live in SoP.md; it is strictly an append-only timeline.

Under the new workflow, when a Deterministic DP executes and triggers the ops/bin/certify protocol, an entry is appended to SoP.md. This entry must follow the strict formatting sequence: ## YYYY-MM-DD HH:MM:SS UTC — DP-OPS-XXXX <summary>.[1] The body must explain the scope and rationale without attempting to serve as proof.[1] Extensive pasting of raw payloads, dump outputs, or code diffs into SoP.md is classified as a drift trigger. If a system policy change is justified by the implementation, that rule must be formally migrated to PoT.md or a docs/ manual; it cannot exist solely as a bullet point in the history ledger.[1]

The Proof Ledger (PoW.md)

Conversely, PoW.md operates exclusively as the proof ledger.[1] It is entirely devoid of narrative justification. Instead, it maintains a rigorous compilation of execution evidence pointers.[1]

When the RESULTS receipt is finalized by the certification binary, the PoW.md entry records the exact pointers to the artifacts proving the execution occurred successfully.[1] This includes the path to the RESULTS document, the OPEN artifact used at the session's genesis, and the DUMP (Contractor Dispatch Dump or Audit Platform Dump) artifact that serialized the state prior to execution.[1]

Furthermore, PoW.md mandates the recording of both positive and negative proofs.[1] If an Investigative DP attempted to validate a specific refactoring path and encountered an irresolvable dependency conflict, that negative proof—the failed checks, ruled-out

hypotheses, and abandoned attempts—must be explicitly documented in the ledger notes.[1] This guarantees that future Integrators do not attempt to recursively traverse known dead ends, thereby maximizing operational efficiency.

System Verification and the Archival Sequence

The strict separation of SoP.md and PoW.md creates a highly resilient audit paradigm. An external auditor or a subsequent AI session can reconstruct the exact state of the system by reading the intent in SoP.md and validating the mathematical reality via the pointers in PoW.md. The integrity of these surfaces is maintained by .github/workflows/sop_policing.yml, which triggers a continuous integration failure if canonical surfaces are changed without corresponding ledger updates.[1]

Additionally, the pruning sequence ensures that active ledgers do not become unwieldy.[1] The ops/bin/prune binary archives older SoP.md and PoW.md entries into archives/surfaces/, maintaining a streamlined active context window while permanently preserving the cryptographic history of the platform.[1] This pruning is executed as the absolute final step of the Mandatory Routing closeout cycle, occurring only after verification, certification, and operator commits have successfully completed.

Staffing Protocol Re-Alignment and Addendum Mechanics

The transition to a JIT Certification model fundamentally clarifies the jurisdictional boundaries of the Stela Staffing Protocol outlined in PoT.md §4.1. Under the deprecated predictive model, the Integrator was frequently forced to act as an execution oracle, blurring the line between drafting constraints and executing logic. The new strategic path forward restores absolute clarity to these roles.[1]

Jurisdictional Isolation

1. **The Operator (Human):** Retains absolute, unassailable authority over final decisions, approvals, branch creation, secret management, and the merging of states into the immutable main trunk.[1] AI agents are strictly forbidden from executing git commits or branch switches.
2. **The Integrator (Lead AI):** Functionally transformed into a structural architect and post-work auditor.[1] The Integrator drafts the deterministic boundaries using ops/bin/draft and verifies the structural integrity of the resulting sidecar and RESULTS artifact against the initial scope.
3. **The Contractor (Guest AI):** Relieved of the burden of making governance decisions. The Contractor strictly executes the specific logic defined in the DP.[1] The Contractor authors the runtime sidecar (CLOSING-*.md) based on observable reality, executes the

verification commands, and runs the certification binary.

The Addendum Authority Protocol

A critical mechanism ensuring that the Contractor never autonomously expands scope is the Addendum Authority Protocol, formalized by the deployment of the ops/bin/addendum binary.[1]

During the mandatory post-work audit, the Integrator reviews the git diff, the RESULTS receipt, and the closing sidecar against the original DP scope definition. If the Integrator identifies that the required scope was exceeded, a boundary condition was not anticipated, or an explicit authorization is needed for work already completed to finalize the packet, a structural intervention is required.[1]

The Integrator must not simply command the Contractor to proceed. Instead, the Integrator utilizes ops/bin/addendum to generate an addendum recommendation as a fully rendered artifact from the canonical ops/src/surfaces/addendum.md.tpl template.[1] The Operator (Human) reviews this artifact and populates the OPERATOR_AUTHORIZATION field. Only then is the authorized addendum handed to the Contractor as a finished, received document. The Contractor does not produce, recommend, or assemble addendum content; it solely executes against the parameters explicitly approved by the Operator within the newly generated addendum boundaries.[1] This guarantees that no execution occurs outside of human-authorized parameters, perfectly aligning with the "System" and "Action" axioms of the Policy of Truth.

Synthesis and Terminal Strategic Directives

The transition from predictive drafting to Certified State Generation represents a mandatory epistemological maturation of the Stela architecture. The temporal paradox—the systemic flaw of forcing an AI Integrator to hallucinate future git states and validation metadata—was the primary catalyst for system drift, failed deployments, and architectural incoherence.

By implementing a rigorous Correct-by-Construction paradigm, the platform permanently eliminates this vulnerability at its source. The strategic path forward rests upon four non-negotiable operational pillars:

1. **Structural Preclusion:** The removal of outcome-dependent Closing Block slots from the dp.md.tpl drafting template ensures that verification metadata cannot be physically invented prior to actual execution.
2. **Typological Segregation:** The strict enforcement of Investigative versus Deterministic Dispatch Packets eliminates conditional logic, preventing the downward delegation of governance authority to executing Contractors and maintaining the sanctity of the active allowlist.
3. **Just-In-Time Certification:** The deployment of the CLOSING-*.md human-authored sidecar and the enhanced ops/bin/certify binary guarantees that all execution receipts are

derived directly from observable, mathematically proven reality at the exact moment of execution completion.

4. **Absolute Canonical Segregation:** The rigid adherence to the Filing Doctrine and Calibrated Canonical Density parameters maintains an impenetrable firewall between runtime binaries (ops/), textual justification (docs/), payload environments (projects/), and immutable state archives (storage/ and archives/).

Execution of this strategy requires absolute discipline. Operators must ruthlessly refuse to authorize mixed-mode packets. Integrators must halt and issue Addendum templates immediately upon encountering ambiguity. Contractors must meticulously update the JIT sidecar based strictly on observable state changes and rely solely on ops/bin/certify for final receipt generation.

By aligning the mechanical workflow seamlessly with the epistemological axioms of the Policy of Truth, the Stela platform guarantees that every recorded action is deterministic, mathematically verifiable, and absolutely true. The resulting architecture is impervious to predictive drift, establishing a self-healing, Correct-by-Construction governance loop uniquely capable of supporting indefinite scale without structural degradation.

Works cited

1. dump-full-main-7567d65ae.txt